Zonoth

SESSION 08 · 2H

Configuration & storage

ConfigMaps, Secrets, volumes, PersistentVolumeClaims and StatefulSets — everything a Pod needs that is not in its image.

ConfigMaps | Secrets | Volumes | PV / PVC / StatefulSets | Lab 08

SESSION 08

What we'll cover today

1 · ConfigMaps

One image, many environments: env vars vs mounted files, update propagation, and immutable ConfigMaps.

2 · Secrets

Types, why base64 is not encryption, encryption at rest, RBAC, and the external stores you'll use in production.

3 · Volumes & persistence

emptyDir to PersistentVolumeClaims, StorageClasses and CSI, then StatefulSets for stateful workloads.

4 · Lab 08

Give Pinboard a real PostgreSQL: a Secret, a ConfigMap, a StatefulSet with a 1 Gi claim — and notes that survive.

01

ConfigMaps

Non-sensitive configuration as a first-class Kubernetes object.

THE PRINCIPLE

Configuration lives outside the image

Build the image **once**, promote the same digest through every environment.

- Different greeting, theme, log format, database URL — same `pinboard-api:1.0`
- Rebuilding to change a setting destroys the "it is the tested artefact" guarantee
- Config in Git, next to the manifests, reviewed like code

ConfigMap

Plain configuration: greetings, feature flags, whole config files. Up to **1 MiB**.

Secret

Same shape, different intent: credentials, tokens, TLS keys. Different RBAC, different handling.

⚠ Neither is a place for megabytes: both live in etcd and are pushed to every node that runs the Pod.

THE OBJECT

Creating a ConfigMap

- `--from-literal`, `--from-file`, `--from-env-file`
- Keys become env var names or file names
- Generate, review, commit — never `create` straight into a cluster

BASH

```
kubectl -n pinboard create configmap pinboard-api-config \
--from-literal=APP_GREETING="Welcome to Pinboard" \
--from-file=nginx.conf \
--dry-run=client -o yaml
```

YAML

```
  apiVersion: v1
kind: ConfigMap
metadata:
  name: pinboard-api-config
  namespace: pinboard
  labels:
    app.kubernetes.io/name: pinboard-api
    app.kubernetes.io/part-of: pinboard
data:
  APP_GREETING: "Welcome to Pinboard on Kubernetes"
  APP_THEME: "emerald"
  LOG_FORMAT: "json"
```

TWO WAYS IN

Environment variables vs mounted files

- `envFrom` — every key becomes an env var; simplest and what Pinboard uses
- `configMapKeyRef` — one key, possibly renamed
- Volume mount — one **file per key** in a directory
- Files win for real config files (nginx, app.yaml) and for live updates

YAML

```yaml
 envFrom:
 - configMapRef:
     name: pinboard-api-config
env:
 - name: APP_THEME
   valueFrom:
     configMapKeyRef:
       name: pinboard-api-config
       key: APP_THEME
volumeMounts:
 - name: config
   mountPath: /etc/pinboard   # one file per key
```

THE GOTCHA

Update propagation

CONSUMED AS	WHAT HAPPENS WHEN THE CONFIGMAP CHANGES
env / envFrom	**Nothing.** Values are a snapshot taken at container start → `kubectl rollout restart`
volume mount	kubelet refreshes the files within about a minute; your app must re-read them
volume with `subPath`	**Never updates.** Avoid `subPath` if you want live config
`immutable: true`	Rejected — delete and recreate. Cheaper on the API server and safe from typos

Common pattern: an annotation holding a checksum of the ConfigMap on the Pod template, so a config change **is** a rollout.

LIVE DEMO

Change a ConfigMap: env vs mounted file

Edit `APP_GREETING` and watch the env-var Pod ignore it until a `rollout restart` — while a mounted file changes under a running container.

What you'll see · the snapshot vs the live file

Files · demos/session08/configmap.yaml · pod-cm-env.yaml · pod-cm-volume.yaml

Duration · 8 min

02

Secrets

The same object with a different name — and a much more careful set of habits.

THE OBJECT

Secrets and their types

TYPE	USED FOR
`Opaque`	Default. Arbitrary key/value — our `pinboard-db` credentials
`kubernetes.io/dockerconfigjson`	Registry pull credentials (`imagePullSecrets`)
`kubernetes.io/tls`	`tls.crt` + `tls.key` for Ingress and Gateway listeners
`kubernetes.io/basic-auth` · `ssh-auth`	Structured credentials with validated keys
`kubernetes.io/service-account-token`	Legacy. Pods get short-lived **projected** tokens instead

✓ Types are mostly a contract for consumers: the type tells a controller which keys must exist.

WORKFLOW

Generate the manifest, don't hand-write base64

```bash
kubectl -n pinboard create secret generic pinboard-db \
--from-literal=POSTGRES_USER=pinboard \
--from-literal=POSTGRES_PASSWORD=pinboard-secret \
--from-literal=POSTGRES_DB=pinboard \
--dry-run=client -o yaml > 01-db-secret.yaml
```

- `--dry-run=client` builds the object locally and never touches the cluster
- `data:` holds base64; `stringData:` takes plain text and is encoded for you on apply
- Other subcommands: `create secret tls`, `create secret docker-registry`

SAY IT OUT LOUD

base64 is encoding, not encryption

```bash
kubectl -n pinboard get secret pinboard-db -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d
# pinboard-secret
```



Anyone with read access

...reads the value. Base64 stops shoulder-surfing, nothing else.



Git is forever

A committed Secret is a leaked Secret, even after the "fix" commit.



etcd and backups

Unencrypted by default — and every etcd snapshot carries them too.

CLOSING THE GAP

Making Secrets actually secret

Encryption at rest

An `EncryptionConfiguration` on the API server encrypts Secrets in etcd — ideally with a **KMS v2** provider, not a local key.

RBAC + namespaces

Nobody gets cluster-wide `get secrets`. Separate namespaces separate blast radius. Audit reads.

Never in Git — plain

If manifests must hold secrets, encrypt them: SOPS + age/KMS, or Sealed Secrets.

Short lives

Rotate. Prefer workload identity and short-lived tokens over long-lived static passwords.

PRODUCTION PATTERN

Keep the source of truth outside the cluster

External Secrets Operator

CRDs `SecretStore` + `ExternalSecret`. The operator reads Vault / AWS Secrets Manager / Azure Key Vault / GCP and **syncs** a normal Kubernetes Secret.

Apps stay unchanged; rotation happens upstream.

Secrets Store CSI Driver

Mounts secrets as a volume at Pod start, straight from the provider. The value never has to exist as a Secret object in etcd.

Optional `secretObjects` sync if a workload needs env vars.

Both keep credentials out of Git and give you one place to rotate. Pick one per platform, not one per team.

MANIFEST

How pinboard-api gets its credentials

- Config by `envFrom`, one secret key by `secretKeyRef`
- The database Pod uses `envFrom:` `secretRef` for the whole Secret
- Env vars leak into `kubectl describe pod`? No — but they do land in crash dumps and logs. Files are safer
- Never `echo $DATABASE_URL` in an entrypoint

YAML

```yaml
  envFrom:
  - configMapRef:
      name: pinboard-api-config
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: pinboard-db
        key: DATABASE_URL
```

From `20-api-deployment.yaml` — the shape you'll apply in the lab.

LIVE DEMO

Create a Secret, then read it straight back

Generate the manifest with `--dry-run=client -o yaml`, apply it, decode the password with `jsonpath` — and see it in a Pod's environment.

What you'll see · stringData vs data, decoding in one line **Files** · demos/session08/secret.yaml **Duration** · 5 min

03

Volumes inside the Pod

Scratch space, injected config, and the one volume type you should almost never use.

THE CATALOGUE

Volume types you will actually meet

emptyDir

Created with the Pod, deleted with it. Scratch, caches, sharing files between containers. `medium: Memory` for tmpfs.

configMap / secret

Keys rendered as read-only files. Secrets land in tmpfs on the node.

projected

Several sources — ConfigMap, Secret, downwardAPI, SA token — into one directory.

downwardAPI

Pod name, namespace, labels, resource limits, as files or env vars.

hostPath

A path on the node. Ties the Pod to one machine and is a well-known escape hatch. **Avoid.**

persistentVolumeClaim

Real, durable storage that outlives the Pod. The rest of this session.

MANIFEST

emptyDir and a projected volume

- `volumes` belong to the Pod, `volumeMounts` to each container
- An emptyDir survives a container **restart**, not a Pod reschedule
- Projected volumes are how the SA token, a CA bundle and config arrive in one directory
- Mount config read-only; set `defaultMode` for keys and certificates

YAML

```yaml
volumes:
- name: cache
  emptyDir: {}
- name: bundle
  projected:
    sources:
      - configMap: { name: pinboard-api-config }
      - secret: { name: pinboard-db }
      - downwardAPI:
          items:
            - path: pod-name
              fieldRef: { fieldPath: metadata.name }
```

04

Persistent storage

PersistentVolumes, claims, StorageClasses — asking for a disk without knowing what a disk is.

THREE OBJECTS

PV, PVC and StorageClass

PersistentVolume

A piece of storage in the cluster — an EBS volume, an NFS export, a local-path directory. **Cluster-scoped**, usually created for you.

PersistentVolumeClaim

A workload's request: size, access mode, class. **Namespaced**, written by the app team, mounted by the Pod.

StorageClass

A named tier — fast SSD, cheap HDD, replicated — plus the provisioner and its parameters. Written by the platform team.

You normally only ever write PVCs. `kubectl get sc` tells you which classes exist and which one is `(default)`. On kind: `standard`, the local-path provisioner.

LIFECYCLE

Provision → bind → use → reclaim

01 — Provision
PVC created → the StorageClass's provisioner makes a volume and a PV

→

02 — Bind
PVC and PV are matched **1:1** and go `Bound`

→

03 — Use
Pod mounts the claim; the volume is attached to that node

→

04 — Reclaim
PVC deleted → `Delete` destroys the volume, `Retain` keeps it for manual recovery

⚠ Deleting a Pod never deletes data — deleting the **PVC** does, if the class reclaims with `Delete`. Most default classes do.

ASK FOR THE RIGHT THING

Access modes and CSI

ACCESS MODE	MEANING
`ReadWriteOnce` **(RWO)**	Read-write by Pods on **one node**. Block storage: EBS, Azure Disk, local-path
`ReadWriteOncePod` **(RWOP)**	Read-write by exactly **one Pod** in the cluster. The safe choice for databases
`ReadOnlyMany` **(ROX)**	Read-only from many nodes
`ReadWriteMany` **(RWX)**	Read-write from many nodes — needs a shared filesystem: NFS, CephFS, EFS, Azure Files

CSI is the plugin interface behind all of it: every vendor ships a driver, and features like snapshots, cloning and volume expansion come with it.

05

StatefulSets & databases

When Pods are not interchangeable: stable names, stable storage, ordered rollouts.

WHAT YOU GET

The four identity guarantees

Stable names

Ordinals, not hashes: `pinboard-db-0`, `-1`, `-2`. A replaced Pod gets the same name back.

Stable DNS

Via the headless Service: `pinboard-db-0.pinboard-db.pinboard.svc.cluster.local` always means that member.

Stable storage

`volumeClaimTemplates` gives each ordinal its own PVC, re-attached to the replacement Pod.

Ordered operations

Created 0→N, deleted and updated N→0, one at a time — unless you set `podManagementPolicy: Parallel`.

MANIFEST

pinboard-db as a StatefulSet

- `serviceName` must name the **headless** Service (session 07)
- One replica: Pinboard needs durability, not high availability
- The claim template creates `data-pinboard-db-0`, 1 Gi, on kind's default class
- The Pod spec adds probes, `PGDATA`, and credentials via `envFrom: secretRef`

YAML

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: pinboard-db, namespace: pinboard }
spec:
  serviceName: pinboard-db      # headless Service
  replicas: 1
  selector:
    matchLabels: { app.kubernetes.io/name: pinboard-db }
  template: { ... }             # postgres:17-alpine + probes
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: ["ReadWriteOnce"]
      resources: { requests: { storage: 1Gi } }
```

THE HONEST ANSWER

Databases on Kubernetes — or managed?

Run it in-cluster when…

- You use a mature **operator** (CloudNativePG, Percona, Strimzi)
- You need the same platform everywhere, on-prem included
- Someone owns backups, restores and version upgrades
- You have tested a restore. Recently.

Use a managed service when…

- The team is small and the database is critical
- You want failover, PITR and patching as a feature
- Storage performance and quorum are not your specialism
- "It's just a StatefulSet" is the only plan

A bare StatefulSet is perfect for learning and for dev environments — which is exactly how we use it in the lab.

LAB 08 · 30–40 MIN

Give Pinboard a real database

Create the credentials Secret and the API ConfigMap, deploy `pinboard-db` as a StatefulSet with a 1 Gi claim and a headless Service, point the API at it with `envFrom` and `DATABASE_URL` — then delete the database Pod and watch your notes come back.

You'll practise: `create secret --dry-run` · ConfigMap `envFrom` · StatefulSet + `volumeClaimTemplates` · PVC inspection · `rollout restart` after a config change

SESSION 08

Key takeaways

✓ One image, injected config

ConfigMaps as env vars need a restart; as files they update live. `immutable: true` when it should never change.

✓ Secrets need real controls

base64 is encoding. Encryption at rest, tight RBAC, nothing in Git, and an external store in production.

✓ Claim storage, don't provision it

PVC + StorageClass + CSI. StatefulSets add stable names, stable DNS and a PVC per ordinal.

✓ Next up

Session 09 — observability, a troubleshooting method, autoscaling and security basics.

Zonoth

Questions?

Up next — Session 09: Operating Kubernetes. Leave the database running, we will break it on purpose.